

LAPORAN PRAKTIKUM
PRAKTIKUM PENGEMBANGAN GAME
PERTEMUAN 5
GAME INSTANCE & SAVEGAME



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
NIU : 535493
Kelas : PL5A1
Dosen Pengampu : Yusron Fuadi, S.Sn., M.Sn.
Asisten : Ezra Bariq Rizqullah / Ikhwan Hanif Firdaus

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Siklus Hidup Objek dan Konsep Game Instance	4
2.2 Mekanisme Penyimpanan Berkas Melalui Kelas SaveGame	4
2.3 Pengelompokan Data Melalui User Defined Structure	5
2.4 Komunikasi Terpisah Menggunakan Blueprint Interface	5
2.5 Eksekusi Penyimpanan Sinkron dan Asinkron	5
2.6 Sistem Deteksi Checkpoint Berbasis Box Collision	6
Hasil dan Pembahasan	7
3.1 Dokumentasi Praktikum	7
3.1.1 Pembuatan Blueprint SaveGame dan Struktur Data Pemain	7
3.1.2 Konfigurasi Blueprint GameInstance pada Pengaturan Proyek	8
3.1.3 Perancangan Blueprint Interface (BPI_SaveData)	9
3.1.4 Implementasi Antarmuka pada BP_GameInstance	10
3.1.5 Inisialisasi Data pada Event Init GameInstance	14
3.1.6 Pemulihan Posisi Karakter pada BP_ThirdPersonCharacter	14
3.1.7 Perancangan Aktor Pemicu Checkpoint (BP_CheckPoint)	15
3.2 Hasil Akhir	17
Kesimpulan	19
Daftar Pustaka	20

Tujuan Praktikum

1. Mahasiswa mampu memahami siklus hidup Game Instance dalam Unreal Engine 5.8 sebagai media penyimpanan data global yang bertahan melintasi pergantian level [1, 2].
2. Mahasiswa mampu membuat kelas SaveGame (BP_SaveGame) serta merancang variabel User Defined Structure (S_PlayerState) untuk membungkus koordinat transformasi pemain secara terstruktur [1, 3, 5].
3. Mahasiswa mampu mendefinisikan dan menerapkan Blueprint Interface (BPI_SaveData) untuk menangani pertukaran data simpan tanpa ketergantungan tipe data langsung (decoupling) [1, 4].
4. Mahasiswa mampu menyusun logika penyimpanan serta pemuatan berkas simpanan secara sinkron maupun asinkron (*asynchronous*) pada Game Instance guna mencegah *frame drop* [1, 9].
5. Mahasiswa mampu membuat aktor pemicu BP_CheckPoint berbasis Box Collision yang bertugas mencatat koordinat transformasi karakter dan memulihkan posisi tersebut saat permainan dimulai kembali [1, 6].

Dasar Teori

2.1 Siklus Hidup Objek dan Konsep Game Instance

Dalam arsitektur Unreal Engine, sebagian besar objek gameplay terikat langsung pada siklus hidup dunia permainan (UWorld) [2]. Ketika pemain berpindah level atau memuat ulang peta, objek seperti GameMode, PlayerController, dan karakter pemain dimusnahkan dari memori lalu dibuat ulang dari awal. Akibatnya, nilai variabel lokal yang tersimpan di dalam aktor tersebut akan tereset ke kondisi baku jika tidak ditampung di tempat lain [8].

Kelas GameInstance dirancang untuk mengatasi karakteristik tersebut. Objek ini dibuat saat mesin game mulai dijalankan dan tetap berada di memori RAM hingga aplikasi game ditutup sepenuhnya oleh pengguna [2]. Game Instance berdiri di luar lingkup level, sehingga posisinya setara dengan konsep *Game Manager* yang umum ditemui pada mesin game Unity [1]. Di dalam urutan inisialisasi mesin, event Init pada Game Instance dieksekusi sebelum event BeginPlay milik level dan aktor mana pun berjalan [8]. Posisi awal ini menjadikannya tempat yang tepat untuk membaca berkas simpanan dari media penyimpanan dan menyiapkan data sesi sebelum dunia permainan ditampilkan ke layar.

2.2 Mekanisme Penyimpanan Berkas Melalui Kelas SaveGame

Penyimpanan data yang bertahan melintasi sesi aplikasi game yang berbeda membutuhkan penulisan data ke dalam media penyimpanan non-volatile (hard disk atau SSD) [3]. Unreal Engine menyediakan kelas dasar SaveGame sebagai wadah serialisasi data [1]. Objek turunan kelas ini menjadi skema penyimpanan variabel yang nantinya dikonversi oleh mesin ke dalam format biner berekstensi .sav di direktori lokal Saved/SaveGames/ [3].

Alur kerja penyimpanan diawali pembentukan objek SaveGame di memori, pengisian variabel yang ingin disimpan, lalu pemanggilan fungsi simpan ke slot tertentu memakai string pengenalan (SlotName) [1, 10]. Keberadaan berkas simpanan divalidasi terlebih dahulu melalui node Does Save Game Exist guna mencegah pembacaan data kosong yang dapat memicu galat *null reference* [3].

2.3 Pengelompokan Data Melalui User Defined Structure

Pada proyek berskala menengah hingga besar, menyimpan data pemain secara terpisah dalam variabel-variabel tunggal membuat graph Blueprint penuh dengan kabel data yang rumit. Unreal Engine menyediakan fitur *User Defined Structure* (Struct) untuk mengelompokkan beberapa variabel terkait ke dalam satu tipe data gabungan [5].

Pada praktikum pertemuan ini, struktur data dinamai `S_PlayerState` sesuai aturan penamaan aset resmi Epic Games [7]. Struktur ini memuat variabel bernama `Transform` bertipe data `Transform` [1]. Tipe data `Transform` menggabungkan tiga komponen vektor sekaligus:

$$\mathbf{T} = \{\mathbf{P}_{\text{Location}}, \mathbf{R}_{\text{Rotation}}, \mathbf{S}_{\text{Scale}}\} \quad (.1)$$

Dengan menyatukan ketiga komponen tersebut ke dalam satu struct, proses pengiriman data posisi pemain antar-Blueprint dapat dilakukan melalui satu pin kabel saja.

2.4 Komunikasi Terpisah Menggunakan Blueprint Interface

Menghubungkan aktor di dalam level dengan Game Instance dapat dilakukan melalui teknik *Casting* (`Cast to BP_GameInstance`). Namun, pemanggilan cast secara langsung menimbulkan ketergantungan kelas yang erat (*tight coupling*) [4]. Jika banyak aktor memanggil cast ke kelas tertentu, aset tersebut akan ikut ditarik ke memori saat level dimuat, yang dapat memperlambat waktu pemuatan [1].

Blueprint Interface merupakan kontrak fungsi abstrak tanpa bodi implementasi [4]. Pada modul 5, antarmuka dinamai `BPI_SaveData` dengan prefix standar `BPI_` [7]. Antarmuka ini mendeklarasikan empat fungsi operasional: `LoadGameData`, `SaveGameData`, `GetGameData`, dan `SavePlayer` [1]. Objek pemanggil cukup memanggil fungsi pesan antarmuka tanpa perlu mengetahui struktur internal kelas target. Jika objek target mengimplementasikan antarmuka tersebut, pesannya akan dieksekusi; jika tidak, panggilan diabaikan tanpa menimbulkan galat pada permainan [4].

2.5 Eksekusi Penyimpanan Sinkron dan Asinkron

Operasi penulisan dan pembacaan berkas dari media penyimpanan fisik membutuhkan waktu pemrosesan I/O yang lebih lambat dibanding akses memori RAM [9]. Jika operasi I/O dijalankan secara sinkron pada utas utama (*game thread*) menggunakan node `Save Game to Slot`, permainan dapat mengalami jeda sesaat (*hitch* atau penurunan *frame rate*) ketika data yang ditulis berukuran besar [3].

Unreal Engine menyediakan alternatif berupa node `Async Save Game to Slot` dan `Async Load Game from Slot` [9]. Node asinkron ini memindahkan proses serialisasi ke

utas latar belakang (*background thread*) dan mengembalikan sinyal keberhasilan lewat pin delegasi `Completed` ketika proses tuntas [1]. Pada rancangan fungsi praktikum ini, sebuah parameter bertipe Boolean bernama `Async` disediakan untuk memberi kebebasan memilih mode eksekusi sesuai kebutuhan skenario permainan.

2.6 Sistem Deteksi Checkpoint Berbasis Box Collision

Checkpoint merupakan mekanisme permainan untuk mencatat kemajuan pemain pada titik tertentu di dalam peta [1]. Deteksi kedatangan karakter ke lokasi checkpoint memanfaatkan komponen tabrakan `Box Collision` [6]. Komponen ini mendeteksi perpotongan volume fisik melalui event `OnComponentBeginOverlap`.

Agar pencatatan posisi tidak dieksekusi berulang kali selama karakter berada di dalam kotak pemicu, alur logika dilindungi oleh node pembatas `Do Once` [1]. Ketika karakter melintasi kotak, koordinat transform karakter diambil melalui node `Get Actor Transform`, lalu dikirimkan ke `Game Instance` melalui pesan antarmuka `Save Player` untuk dituliskan ke berkas penyimpanan [1].

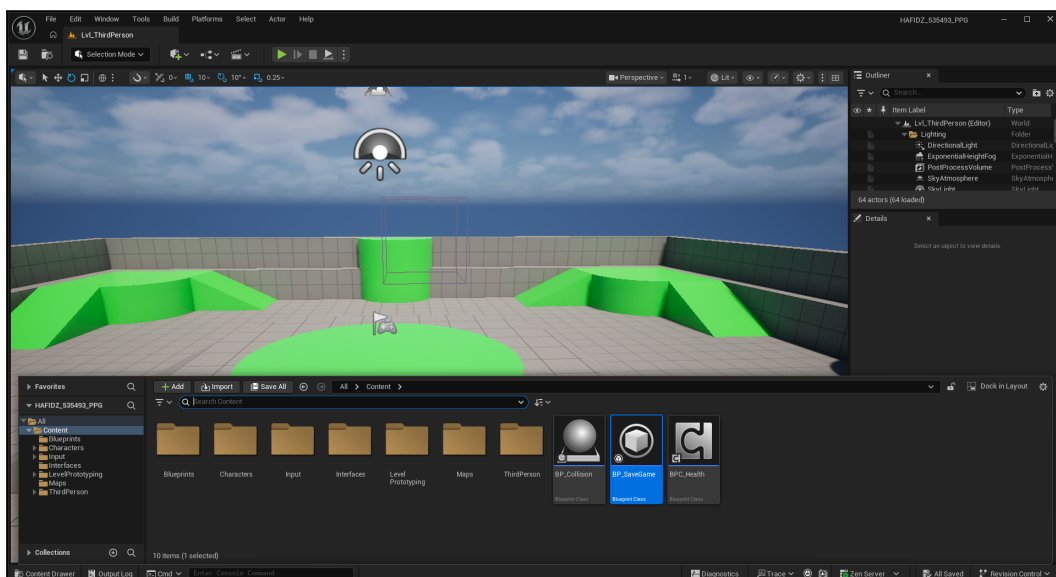
Hasil dan Pembahasan

Bab ini mendokumentasikan pelaksanaan praktikum Pertemuan 5 dengan topik Game Instance dan SaveGame. Praktikum saya kerjakan pada proyek Unreal Engine 5.8 HAFIDZ_535493_PPG menggunakan template Third Person. Seluruh tahapan disajikan berurutan sesuai alur modul praktikum beserta keterangan teknis pada setiap gambar.

3.1 Dokumentasi Praktikum

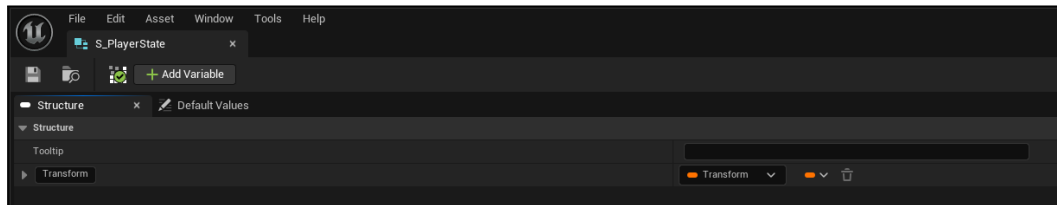
3.1.1 Pembuatan Blueprint SaveGame dan Struktur Data Pemain

1. **Pembuatan kelas SaveGame (BP_SaveGame).** Melalui Content Drawer, menu penambahan aset Blueprint Class dibuka. Pada jendela *Pick Parent Class*, bagian *ALL CLASSES* ditelusuri dengan kata kunci pencarian save. Kelas dasar **SaveGame** dipilih sebagai kelas induk, kemudian berkas disimpan dengan nama **BP_SaveGame**. Berkas ini berfungsi menampung data permainan yang akan diserialisasi ke media penyimpanan lokal.



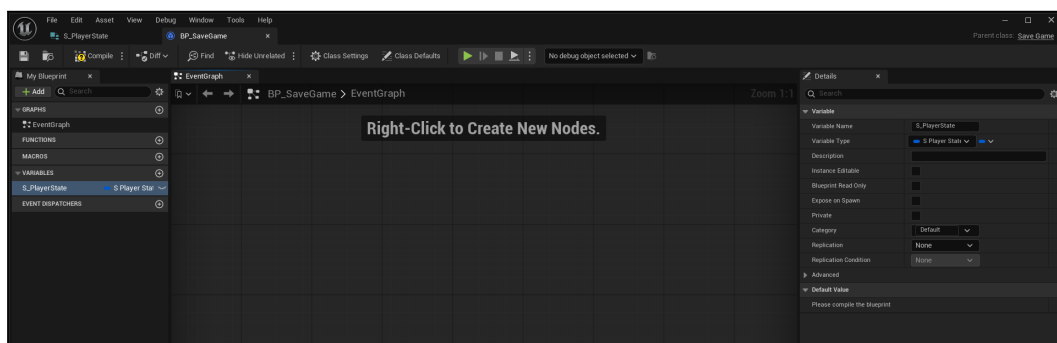
Gambar 1: Pemilihan kelas induk SaveGame pada jendela Pick Parent Class untuk membuat aset BP_SaveGame.

2. **Pembuatan User Defined Structure (S_PlayerState).** Pada Content Drawer, klik kanan dilakukan untuk membuka menu konteks **Blueprint > Structure**. Struktur data baru dibuat dan diberi nama **S_PlayerState**. Tab editor struktur kemudian dibuka, dan satu variabel baru ditambahkan dengan nama **Transform** serta tipe data **Transform**. Konfigurasi ini menyatukan parameter lokasi, rotasi, dan skala karakter ke dalam satu kesatuan data.



Gambar 2: Struktur data S_PlayerState dengan variabel Transform bertipe data Transform.

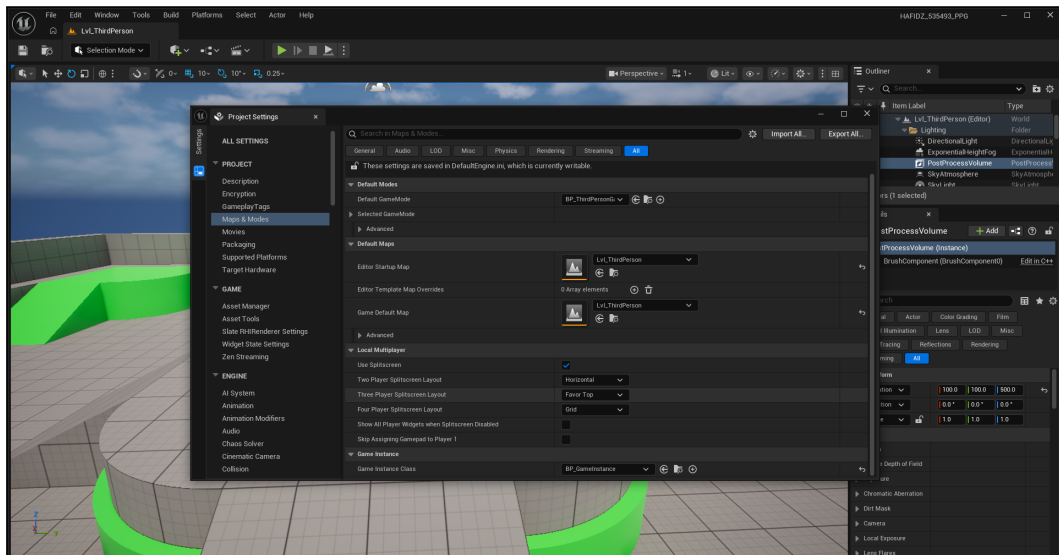
3. **Penambahan variabel struktur ke dalam BP_SaveGame.** Berkas BP_SaveGame dibuka pada editor Blueprint. Pada panel *My Blueprint*, tombol tambah variabel ditekan untuk membuat variabel baru bernama **S_PlayerState**. Tipe data variabel diatur mengarah ke struktur **S_PlayerState** yang baru saja dibuat. Melalui langkah ini, berkas simpanan memiliki penampung untuk data posisi pemain.



Gambar 3: Variabel S_PlayerState bertipe data struktur S_PlayerState pada panel My Blueprint BP_SaveGame.

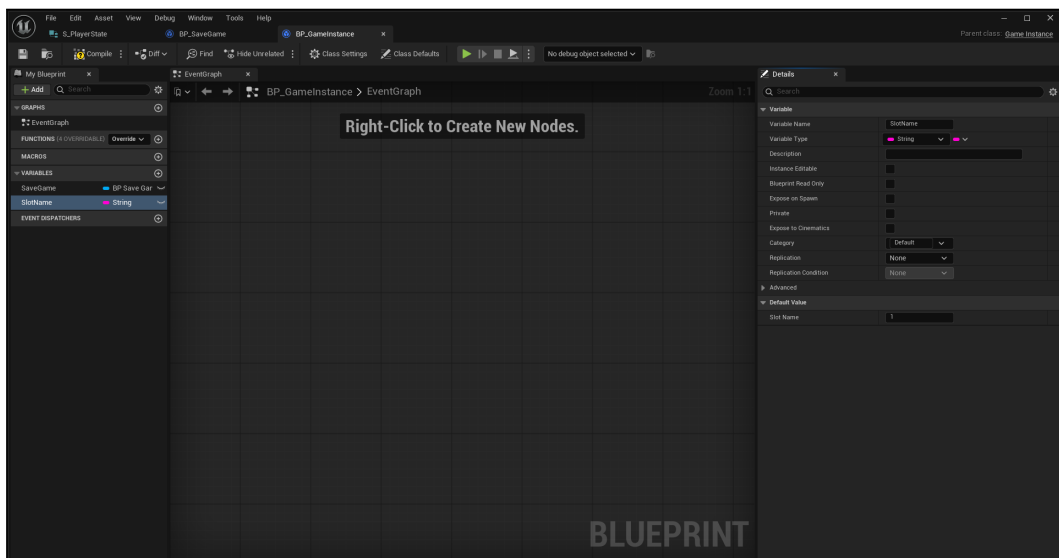
3.1.2 Konfigurasi Blueprint GameInstance pada Pengaturan Proyek

1. **Pembuatan BP_GameInstance dan registrasi pada Project Settings.** Aset Blueprint baru dibuat dengan memilih kelas induk **GameInstance** dan dinamai **BP_GameInstance**. Agar mesin game mengenali kelas ini sebagai pengelola instansi global utama, menu **Edit > Project Settings > Project - Maps & Modes** dibuka. Pada seksi *Game Instance*, opsi **Game Instance Class** diganti dari kelas bawaan mesin menjadi **BP_GameInstance**.



Gambar 4: Pembuatan BP_GameInstance dan pengaturannya pada menu Project Settings bagian Maps & Modes.

2. **Deklarasi variabel SaveGame dan SlotName pada Game Instance.** Editor BP_GameInstance dibuka. Dua variabel dibuat pada panel My Blueprint: variabel **SaveGame** bertipe referensi objek **BP_SaveGame**, serta variabel **SlotName** bertipe data **String**. Pada panel Details variabel SlotName, nilai bawaan (*Default Value*) diisi dengan string "1" sebagai pengenal slot berkas penyimpanan lokal.



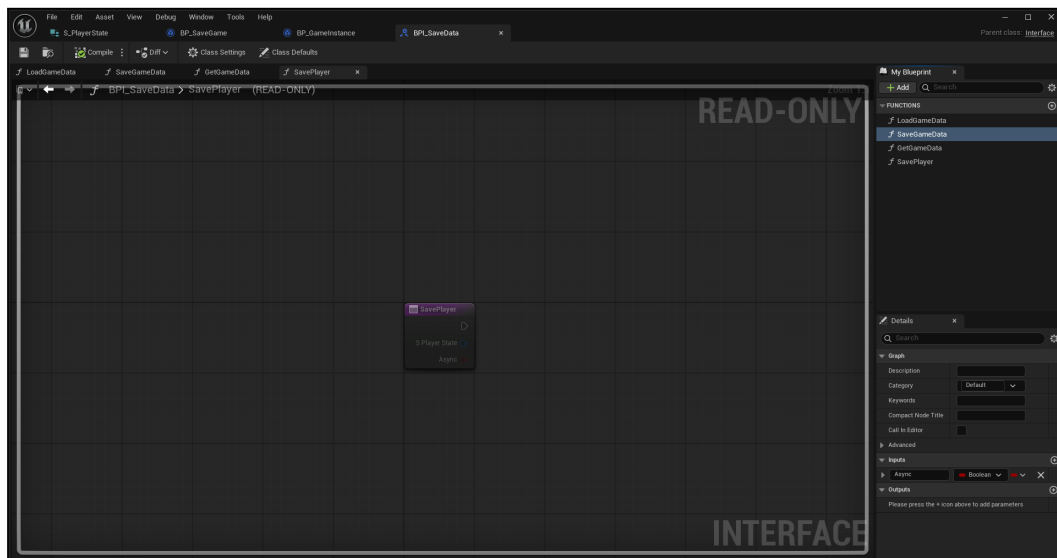
Gambar 5: Panel variabel BP_GameInstance yang memuat referensi objek SaveGame dan SlotName dengan nilai default 1.

3.1.3 Perancangan Blueprint Interface (BPI_SaveData)

1. **Mendefinisikan fungsi dan parameter antarmuka BPI_SaveData.** Pada direktori konten, aset Blueprint Interface dibuat dengan nama **BPI_SaveData**. Empat fungsi

operasional dideklarasikan di dalamnya beserta parameter masukan dan keluaran yang presisi:

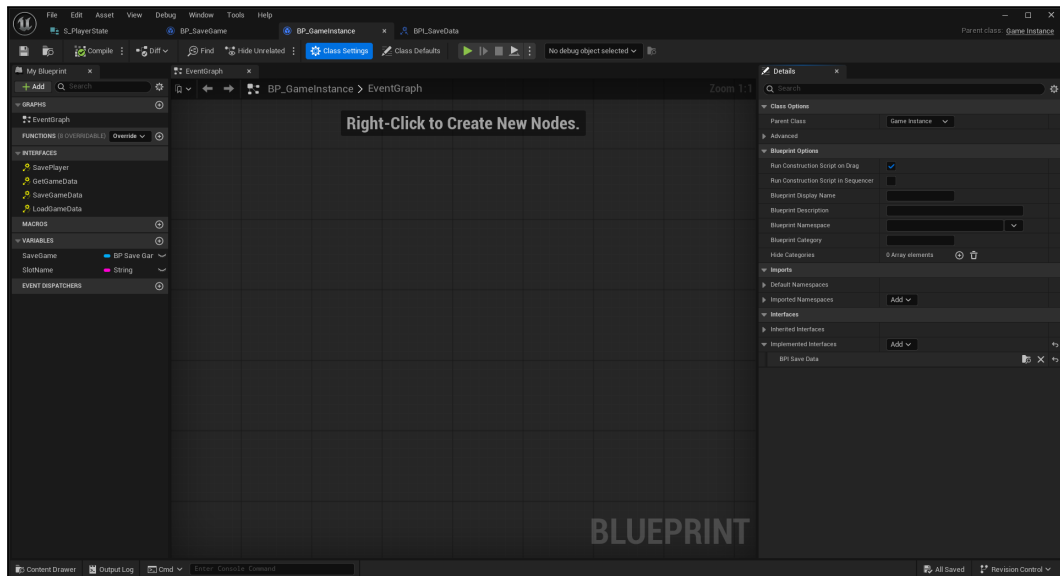
- **LoadGameData:** memiliki parameter input Async bertipe Boolean.
- **SaveGameData:** memiliki parameter input Async bertipe Boolean.
- **GetGameData:** memiliki parameter output S_PlayerState bertipe data struktur S_PlayerState.
- **SavePlayer:** memiliki parameter input S_PlayerState (tipe struktur S_PlayerState) dan parameter input Async (tipe Boolean).



Gambar 6: Daftar empat fungsi pada antarmuka BPI_SaveData beserta konfigurasi parameter input dan output.

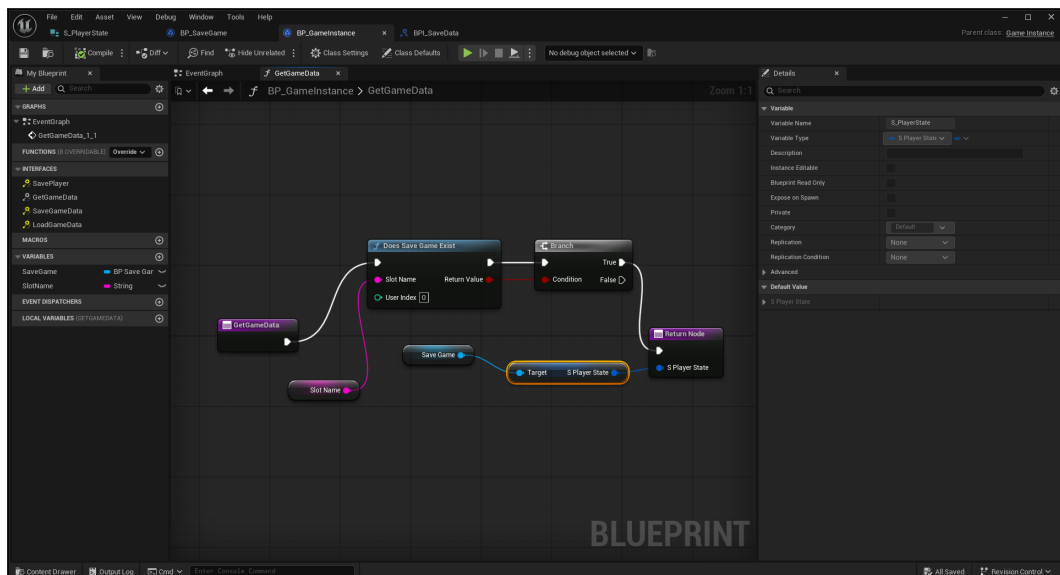
3.1.4 Implementasi Antarmuka pada BP_GameInstance

1. **Penerapan BPI_SaveData pada Class Settings.** Berkas BP_GameInstance dibuka kembali, lalu tombol **Class Settings** pada toolbar atas diklik. Pada panel Details bagian **Interfaces > Implemented Interfaces**, tombol **Add** ditekan untuk memasukkan **BPI_SaveData**. Setelah Blueprint dikompilasi, keempat fungsi antarmuka muncul di bawah kategori **INTERFACES** pada panel My Blueprint.



Gambar 7: Implementasi antarmuka BPI_SaveData melalui Class Settings pada BP_GameInstance.

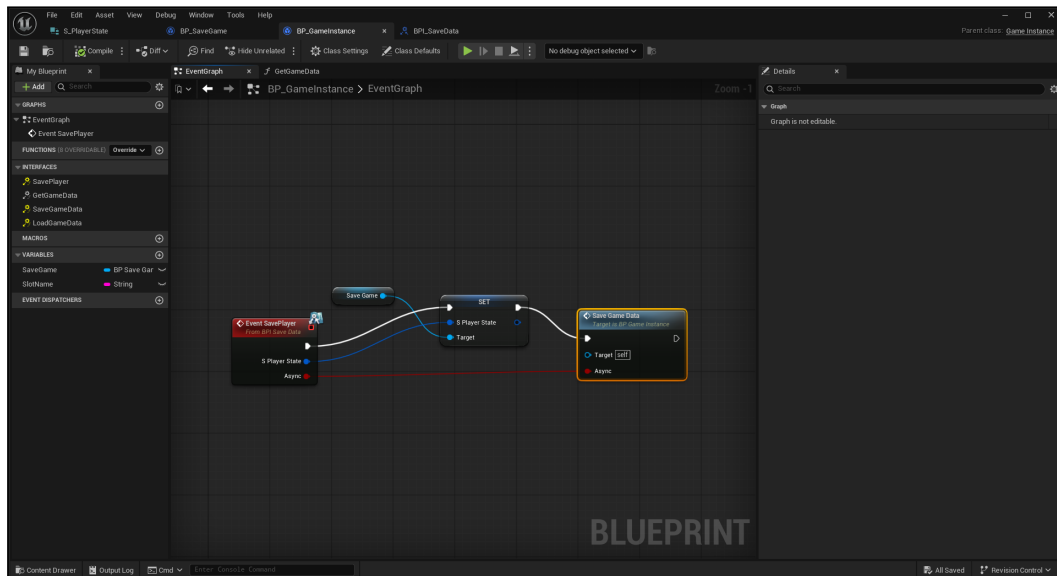
2. **Menyusun logika fungsi GetGameData.** Fungsi GetGameData dibuka untuk menyusun alur pengambilan data posisi. Node **Does Save Game Exist** dipasang dengan masukan SlotName dan User Index 0 untuk memeriksa keberadaan berkas di penyimpanan. Hasil pemeriksaan disambungkan ke node **Branch**. Jika bernilai benar (True), nilai S_PlayerState diambil dari variabel SaveGame dan dikembalikan melalui **Return Node**. Jika salah (False), Return Node mengembalikan koordinat default dengan posisi (0, 0, 0) dan skala (1, 1, 1).



Gambar 8: Rangkaian graph fungsi GetGameData: validasi keberadaan berkas simpanan dan pengembalian data posisi.

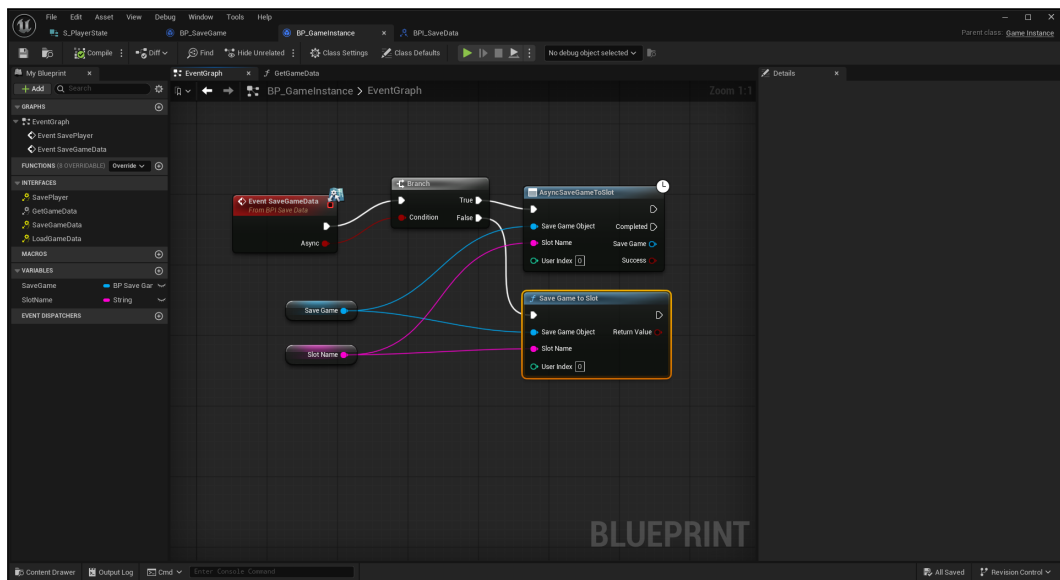
3. **Menyusun logika fungsi SavePlayer.** Fungsi SavePlayer dibuka untuk menerima

data terbaru karakter. Node event menerima struktur `S_PlayerState` dan boolean `Async`. Alur graph mengeksekusi node **SET S_PlayerState** pada objek variabel `SaveGame` yang ada di memori. Selanjutnya, alur diteruskan untuk memanggil fungsi **SaveGameData** dengan membawa nilai boolean `Async` tersebut.



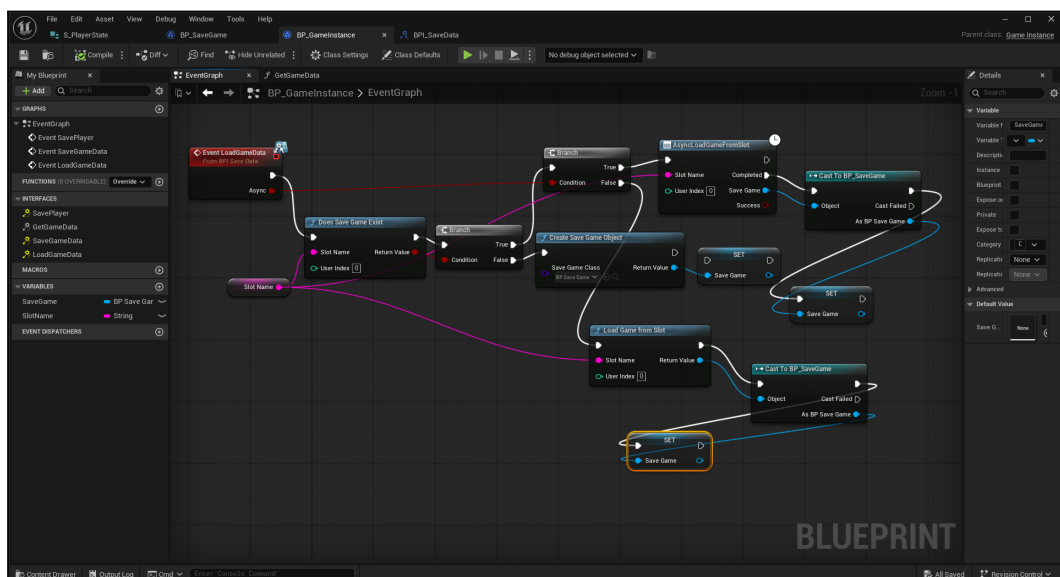
Gambar 9: Graph fungsi `SavePlayer`: memperbarui nilai struktur pada objek `SaveGame` lalu memanggil fungsi `SaveGameData`.

4. **Menyusun logika fungsi `SaveGameData`.** Logika fungsi `SaveGameData` dibuka untuk menangani proses penulisan ke media penyimpanan lokal. Parameter boolean `Async` disambungkan ke node **Branch**. Jika `True`, node **Async Save Game to Slot** dipanggil menggunakan masukan objek `SaveGame` dan `SlotName`. Jika `False`, penyimpanan dieksekusi secara sinkron melalui node standar **Save Game to Slot**.



Gambar 10: Graph fungsi SaveGameData: percabangan penyimpanan berkas mode asinkron atau sinkron ke slot.

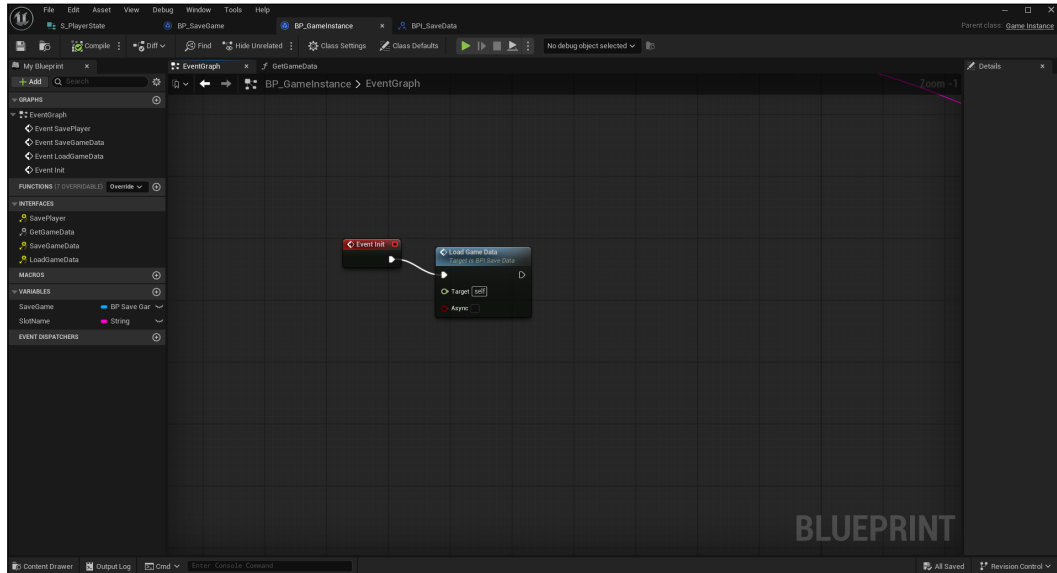
5. **Menyusun logika fungsi LoadGameData.** Logika fungsi LoadGameData bertugas memuat berkas simpanan ke memori RAM. Graph diawali dengan memeriksa keberadaan berkas menggunakan **Does Save Game Exist**. Jika berkas ada (True), percabangan memeriksa opsi Async untuk memanggil **Async Load Game from Slot** atau **Load Game from Slot**. Objek hasil pemuatan di-cast ke **BP_Savegame** lalu disimpan ke variabel SaveGame. Jika berkas belum ditemukan (False), alur memanggil node **Create Save Game Object** dari kelas BP_SaveGame dan menetakannya ke variabel SaveGame sebagai data awal.



Gambar 11: Graph fungsi LoadGameData: pembacaan berkas dari disk atau pembuatan objek simpanan baru jika belum ada.

3.1.5 Inisialisasi Data pada Event Init GameInstance

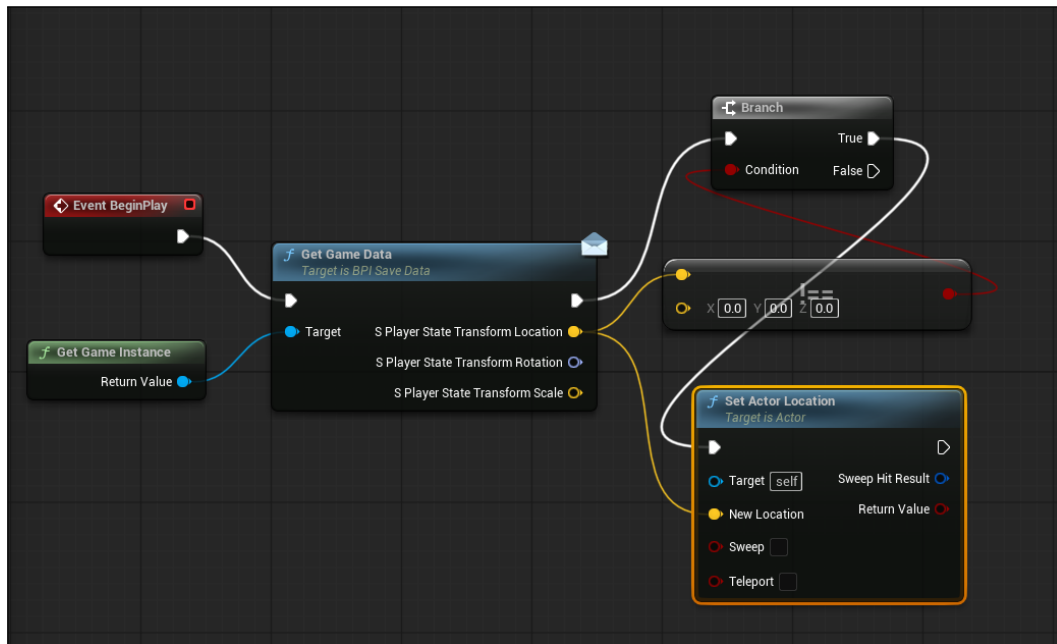
1. **Menghubungkan Event Init ke pemanggilan LoadGameData.** Pada Event Graph utama milik BP_GameInstance, node **Event Init** diletakkan. Pin eksekusi Event Init disambungkan langsung ke pemanggilan fungsi **Load Game Data** dengan target *self* dan parameter Async dibiarkan nonaktif. Pengaturan ini memastikan berkas simpanan langsung dibaca saat game pertama kali dibuka sebelum level dan aktor dimuat.



Gambar 12: Eksekusi pemuatan data awal melalui Event Init pada Event Graph BP_GameInstance.

3.1.6 Pemulihan Posisi Karakter pada BP_ThirdPersonCharacter

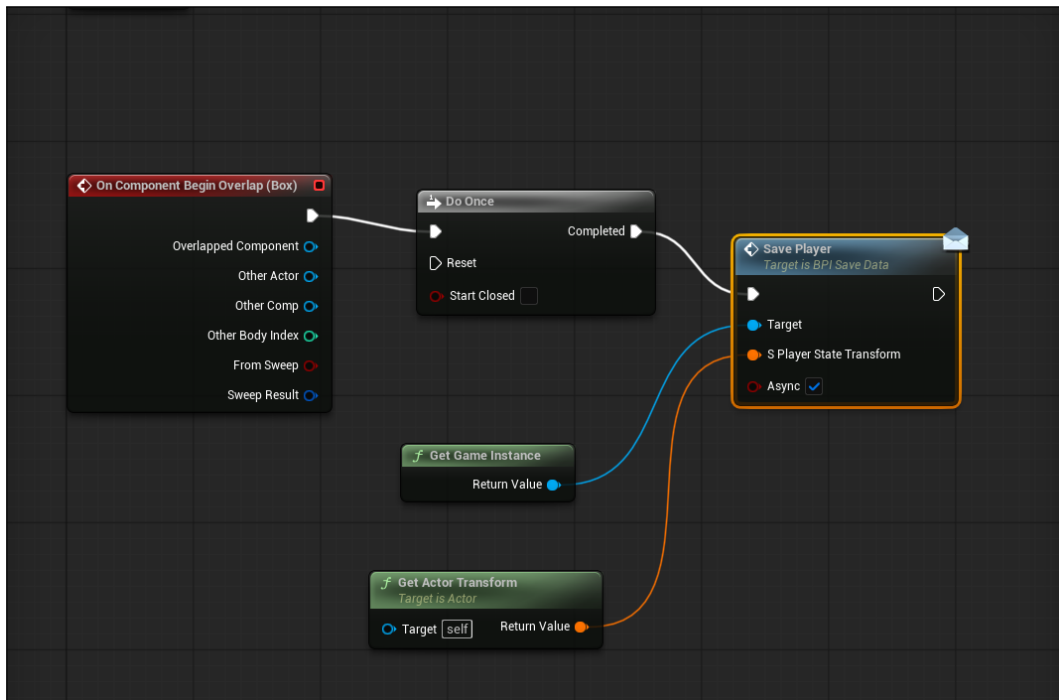
1. **Membaca data simpanan dan memperbarui posisi pada BeginPlay.** Karakter BP_ThirdPersonCharacter dibuka pada editor Blueprint. Pada Event Graph, node **Event BeginPlay** disambungkan ke node panggilan pesan antarmuka **Get Game Data** dengan target dari node **Get Game Instance**. Struktur keluaran diurai untuk mengambil nilai koordinat lokasi. Sebuah node **Branch** memeriksa apakah koordinat lokasi tidak bernilai nol (0, 0, 0). Jika True, node **Set Actor Location** dijalankan untuk memindahkan karakter ke posisi simpanan tersebut.



Gambar 13: Logika BeginPlay pada karakter pemain untuk memulihkan koordinat lokasi dari Game Instance.

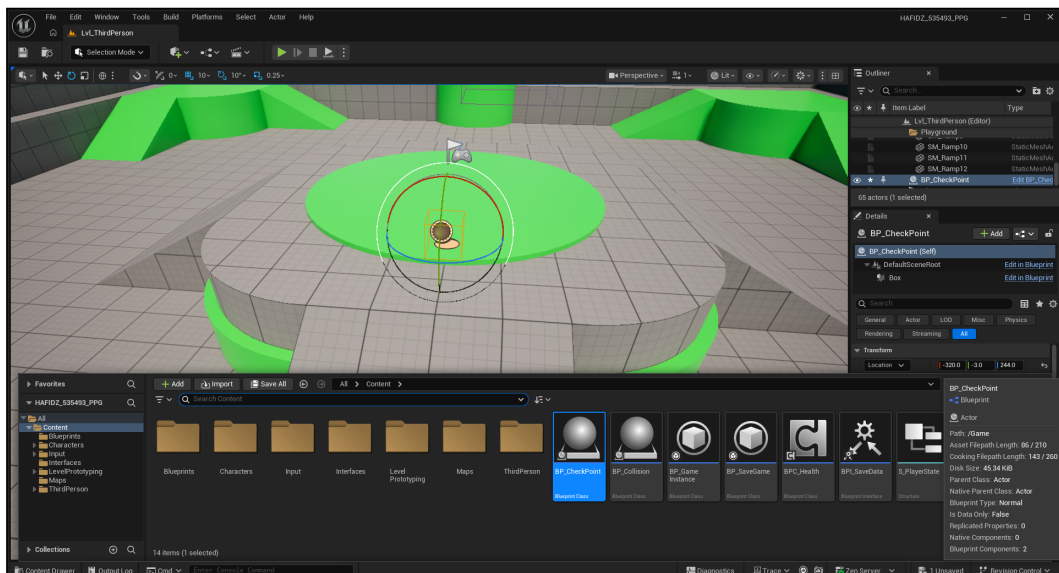
3.1.7 Perancangan Aktor Pemicu Checkpoint (BP_CheckPoint)

1. **Merancang komponen dan logika pemicu pada BP_CheckPoint.** Aktor Blueprint baru dibuat dengan nama **BP_CheckPoint**. Di dalam hierarki komponen, komponen **Box Collision** bernama Box ditambahkan. Pada Event Graph, event **On Component Begin Overlap (Box)** disambungkan ke node **Do Once** agar pencatatan hanya terjadi satu kali saat dilewati. Selanjutnya, node **Get Actor Transform** mengambil posisi checkpoint, dan node **Get Game Instance** menjadi target panggilan pesan antarmuka **Save Player** dengan opsi Async diaktifkan.



Gambar 14: Komponen Box Collision dan alur Overlap pada BP_CheckPoint yang memanggil interface Save Player.

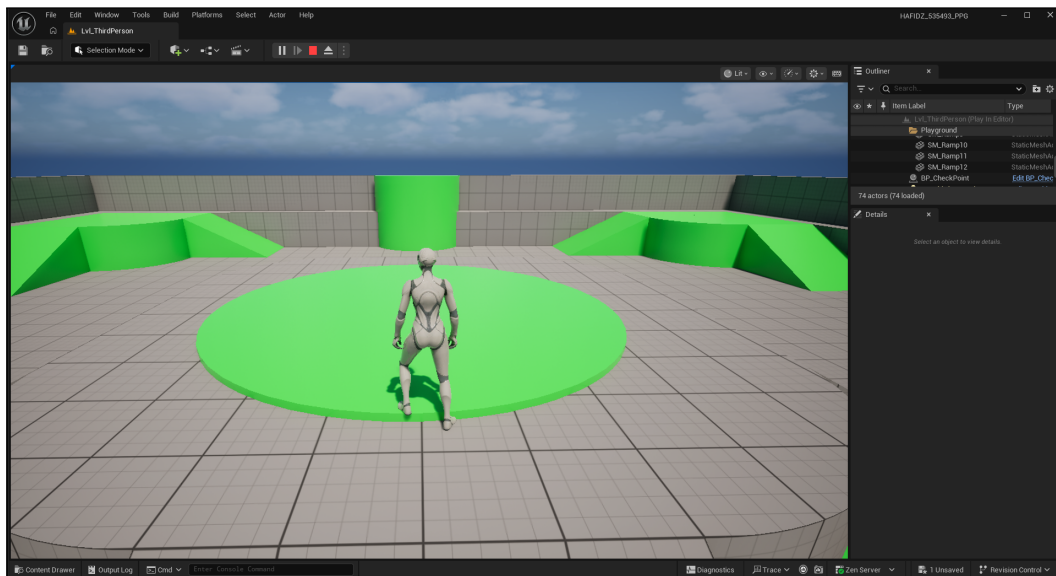
2. **Menempatkan instance BP_CheckPoint pada level.** Aktor BP_CheckPoint diseret dari Content Drawer ke dalam viewport level permainan. Checkpoint diletakkan di tengah lintasan pada posisi koordinat tertentu di depan area awal pemain (PlayerStart) agar dapat dilalui secara langsung oleh karakter saat berjalan.



Gambar 15: Penempatan instance aktor BP_CheckPoint pada viewport level permainan.

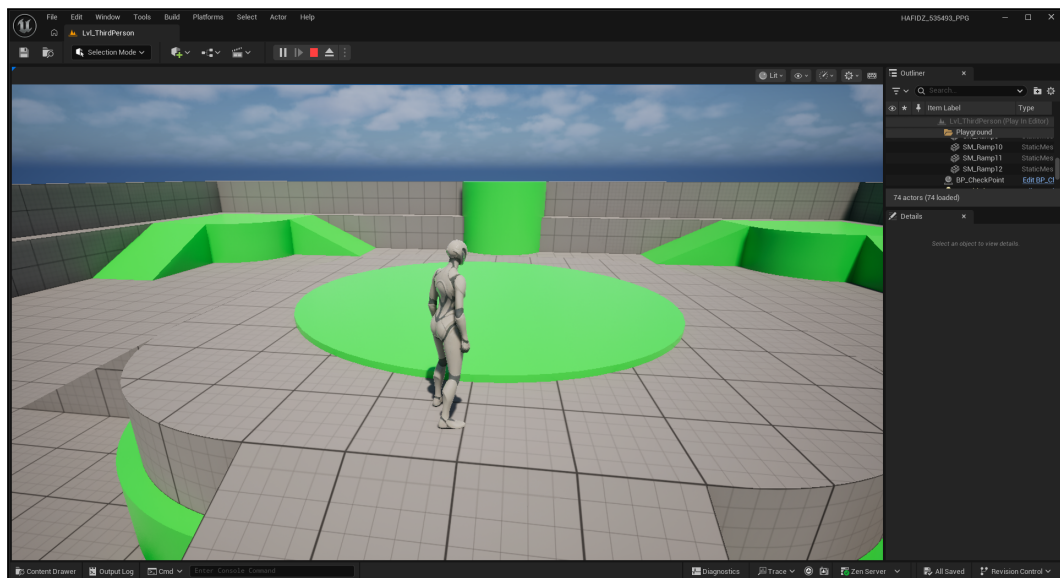
3.2 Hasil Akhir

Pengujian sistem dilakukan langsung di dalam editor level Unreal Engine 5.8 dengan menekan tombol **Play**. Pada tahap awal pengujian, karakter Third Person muncul di titik awal standar yang ditentukan oleh aktor PlayerStart. Karakter kemudian digerakkan melintasi rintangan menuju ke arah volume kawat BP_CheckPoint. Saat karakter menyentuh batas tabrakan, event overlap terpicu. Node Do Once memastikan event hanya ditembakkan satu kali, mengambil koordinat transform tempat karakter berada, lalu mengirimkan data tersebut ke Game Instance melalui fungsi pesan Save Player secara asinkron. Berkas 1.sav berhasil diperbarui di media penyimpanan lokal tanpa ada penurunan kestabilan visual pada viewport.



Gambar 16: Karakter berjalan melintasi BP_CheckPoint dan memicu penyimpanan data posisi pemain.

Tahap kedua pengujian dilakukan untuk memverifikasi kemampuan pemulihan data simpanan (*persistence*). Simulasi permainan dihentikan dengan menekan tombol **Stop**, kemudian tombol **Play** ditekan kembali untuk memulai sesi permainan baru. Saat inisialisasi berjalan, Game Instance mengeksekusi Event Init dan memuat isi berkas slot "1" ke dalam memori. Ketika level dimuat, node Event BeginPlay pada BP_ThirdPersonCharacter meminta data posisi via Get Game Data. Karena koordinat yang tersimpan bukan bernilai nol, node Set Actor Location memindahkan karakter langsung ke titik koordinat checkpoint terakhir. Karakter berhasil berdiri di lokasi checkpoint tersebut tanpa harus mengulang dari titik PlayerStart, membuktikan integrasi Game Instance dan SaveGame telah berjalan sesuai instruksi modul praktikum [1].



Gambar 17: Hasil uji coba play ulang: karakter otomatis dimuat pada posisi checkpoint terakhir dari berkas SaveGame.

Kesimpulan

Dari praktikum Pertemuan 5 mengenai Game Instance dan SaveGame ini, saya memahami bagaimana Unreal Engine mengelola data persisten yang melintasi siklus hidup level maupun sesi aplikasi game. Kelas BP_GameInstance menjadi wadah penyimpanan data global yang aman karena tetap berada di memori RAM selama permainan berjalan, berbeda dengan aktor level yang dihancurkan setiap kali peta dimuat ulang.

Penggunaan kelas dasar BP_SaveGame dan struktur data S_PlayerState memberikan alur penyimpanan yang rapi. Mengelompokkan koordinat lokasi, rotasi, dan skala ke dalam satu variabel Transform menyederhanakan kabel graph Blueprint. Pengecekan awal dengan node Does Save Game Exist juga mencegah terjadinya galat saat membaca slot berkas penyimpanan yang belum ada. Selain itu, pemanfaatan operasi asinkron terbukti menjaga jalannya game tetap mulus tanpa jeda pembekuan frame saat menulis berkas ke media penyimpanan.

Penerapan antarmuka BPI_SaveData pada Game Instance kembali memperkuat arsitektur modular tanpa ketergantungan casting langsung. Aktor pemicu BP_CheckPoint dan karakter pemain dapat saling bertukar data simpanan melalui fungsi pesan antarmuka secara bersih. Hasil pengujian menunjukkan bahwa posisi karakter di checkpoint berhasil ditulis ke berkas lokal 1.sav dan langsung dipulihkan saat permainan dijalankan ulang, membuktikan seluruh mekanisme telah bekerja sesuai panduan modul praktikum [1].

Daftar Pustaka

- [1] Modul Praktikum Pengembangan Game Pertemuan 5: Game Instance & SaveGame, T.A. 2026/2027. Semester Gasal, Universitas Gadjah Mada.
- [2] Epic Games, “GameInstance in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/game-instance-in-unreal-engine>.
- [3] Epic Games, “Saving and Loading your Game in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/saving-and-loading-your-game-in-unreal-engine>.
- [4] Epic Games, “Blueprint Interface in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-interface-in-unreal-engine>.
- [5] Epic Games, “Structs in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/structs-in-unreal-engine>.
- [6] Epic Games, “Collision in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/collision-in-unreal-engine>.
- [7] Epic Games, “Recommended Asset Naming Conventions in Unreal Engine Projects”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/recommended-asset-naming-conventions-in-unreal-engine-projects>.
- [8] Unreal Engine Community Forum, “How to check which Begin Play or something similar goes first”, 2024. [Online]. Tersedia: <https://forums.unrealengine.com/t/how-to-check-which-begin-play-or-something-similar-goes-first-c/1565114/2>.
- [9] Epic Games, “Asynchronous Loading in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/asynchronous-loading-in-unreal-engine>.
- [10] Smart Poly, “UE5 Save Game Explained Simply”, 2023. [Online Video]. Tersedia: https://www.youtube.com/watch?v=b60z6fb_mI0.